1. Abstract

Digit Studio is a browser-based application that recognizes handwritten digits (0–9) in real time. The user draws a digit on an HTML canvas; the image is sent to a Flask back end, converted into an 8 × 8 feature grid that matches the format of the scikit-learn digits dataset, and classified by a machine-learning model of the user's choice. The interface shows the predicted digit, a confidence score, a probability bar for every digit, and the 8 × 8 input the model actually saw.

Three classifiers were trained and compared: a Support Vector Machine (SVM), a Random Forest and a Multi-Layer Perceptron (MLP) neural network. Hyper-parameters were tuned with 5-fold cross-validation and the models were evaluated on a held-out test set and on a set of stroke-thickness variations. The SVM achieved the best test accuracy of 99.44%. The project is packaged with Docker so it can be deployed on any container platform.

2. Introduction

Handwritten digit recognition is a classic computer-vision problem with practical uses such as reading postal codes, bank cheques and form entries. It is also an ideal learning problem because the images are small and the labels are unambiguous. Most tutorials stop at reporting an accuracy number inside a notebook. This project goes further by turning the trained models into an interactive product that anyone can use from a browser.

2.1 Objectives

- Train and compare three different classifiers on handwritten digit data.
- Build a preprocessing pipeline that makes free-hand canvas drawings look like dataset images.
- Improve robustness to thick and thin pen strokes, shifts, rotation and scaling using data augmentation.
- Provide a clean web interface with live prediction, per-digit confidence and model switching.
- Present model results (accuracy, confusion matrix, PCA) inside the application.
- Package the application with Docker for repeatable deployment.

2.2 Technologies Used

Component	Technology
Language	Python 3.11, JavaScript, HTML/CSS
Web framework	Flask 3.0.3 served with Gunicorn 22.0.0
Machine learning	scikit-learn 1.5.2 (SVC, RandomForestClassifier, MLPClassifier, GridSearchCV, PCA)
Numerical / image	NumPy (below 2.0), Pillow 10.4.0
Visualization	Matplotlib 3.9.2

Component	Technology
Model storage	joblib 1.4.2
Deployment	Docker (python:3.11-slim base image)

3. System Overview

The application has two parts: an offline training stage and an online prediction stage. Running train.py fits all models and saves them to disk together with their metrics and the two result charts. The Flask application loads these artifacts once at start-up and uses them to answer prediction requests.

3.1 Prediction Workflow

- The user draws a digit on the 280 × 280 canvas using the brush (adjustable width).
- The browser sends the canvas as a base64 PNG data URL, together with the selected model name, to the /predict endpoint.
- The server validates the image (PNG format, size and dimension limits) and converts it to grayscale.
- The preprocessing function crops, centres, resizes and normalizes the drawing into 64 features.
- The chosen model returns class probabilities; the server responds with the predicted digit, the probabilities (in percent) and the 8 × 8 pixel grid.
- The interface updates the large digit, the confidence ring, the bar chart and the input preview.

3.2 API Endpoints

Route	Method	Purpose
/	GET	Renders the main page with model metrics (index.html).
/predict	POST	Accepts JSON {image, model}; returns prediction, probabilities, pixels and model name.
/health	GET	Returns {"status": "ok"} for health checks.

4. Dataset

The project uses the digits dataset bundled with scikit-learn. It contains 1,797 grayscale images of handwritten digits, each 8 × 8 pixels with intensity values from 0 to 16, and roughly 180 samples per class. The data was divided into 80% training (1,437 images) and 20% test (360 images) using a stratified split with random_state = 42, so every digit is represented in the same proportion in both sets.

Figure 1 (left chart) shows a two-dimensional PCA projection of the standardized features. Some digits, such as 0, 4 and 6, form fairly distinct groups, while others overlap heavily. This overlap explains why a non-linear classifier is needed.

5. Image Preprocessing

A drawing on a 280 × 280 canvas looks very different from the 8 × 8 images the models were trained on. The shared module digit_preprocessing.py therefore converts both training digits and user drawings with the same function, which keeps the training and live data consistent.

- **Invert and grayscale:** the canvas PNG is converted to grayscale and inverted so ink is bright and the background is zero.
- **Crop:** pixels brighter than 40 are treated as ink and the image is cropped to their bounding box.
- **Pad to a square:** the crop is centred on a black square whose side is 1.4 times the larger dimension, leaving a margin like the original dataset.
- **Resize:** the square is reduced to 8 × 8 pixels with Lanczos resampling.
- **Normalize:** values are scaled so the brightest pixel equals 16, matching the dataset range, then flattened to 64 features.

Empty or nearly empty canvases (fewer than 30 ink pixels) are rejected with the message “Please draw a digit first”.

6. Model Training

6.1 Classifiers and Tuning

Each classifier is wrapped in a scikit-learn Pipeline with a StandardScaler, so feature scaling is learned only from training data. GridSearchCV with 5-fold cross-validation selected the hyper-parameters below.

Model	Search grid	Best parameters
SVM (RBF)	C ∈ {1, 10}; gamma ∈ {scale, 0.001}	C = 10, gamma = scale, kernel = rbf
Random Forest	n_estimators ∈ {100, 300}; max_depth ∈ {None, 15}	n_estimators = 100, max_depth = None
Neural Network (MLP)	hidden layers ∈ {(64,), (128, 64)}; alpha ∈ {1e-4, 1e-2}	hidden layers = (128, 64), alpha = 0.01

6.2 Data Augmentation

Users draw with a thick mouse brush, whereas the original digits are thin and neatly centred. To close this gap, every training image was kept in its original form and four augmented copies were added (stroke modes: normal, thick, thin, normal). Each copy is upscaled to 280 × 280 and randomly:

- rotated by −10° to +10°;
- shifted by up to 14 pixels horizontally and vertically;
- scaled by a factor between 0.88 and 1.12;
- thickened with a max filter (kernel 23–47) or thinned with a min filter (kernel 3–5).

After the best parameters were found on the original training data, each model was refitted on the augmented set (five times larger than the original). The test set itself was not augmented for the headline accuracy, so there is no leakage.

6.3 Evaluation Metrics

- Cross-validation accuracy: mean 5-fold accuracy on the original training data.
- Test accuracy: accuracy on the 360 untouched test images.
- Variation (robustness) accuracy: average accuracy on three computer-generated versions of the test images (thick, thin and normal strokes with random shifts and rotations).

The best model by test accuracy is saved as model.joblib, and all three models are saved together in models.joblib so the interface can switch between them.

7. Results

Model	CV accuracy	Test accuracy	Variation accuracy
SVM (best)	97.29%	99.44%	97.22%
Random Forest	96.66%	98.06%	96.30%
Neural Network (MLP)	96.94%	98.89%	96.67%

Table 1: Model comparison (from metrics.json)

The SVM was the most accurate on every measure. On the 360 test images it made only two mistakes, a “2” predicted as “1” and a “5” predicted as “3”, which gives 358/360 = 99.44%. All models lose about two percentage points when stroke style changes, which shows that the variation test is harder than the standard test, and that real hand-drawn input remains more difficult than the clean dataset.

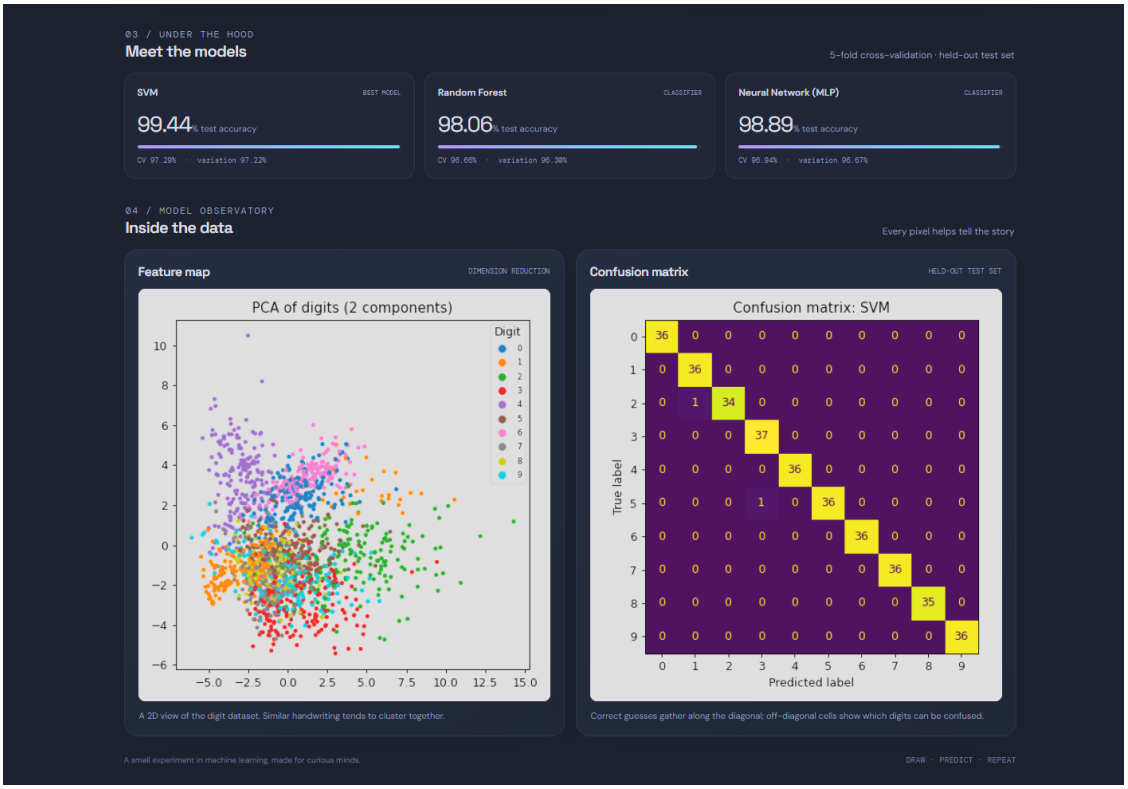


Figure 1: "Meet the models" cards, PCA feature map and SVM confusion matrix, as shown in the application

The confusion matrix has a strong diagonal, so almost all test images are classified correctly. Off-diagonal cells show the only confusions. The application displays the SVM matrix because it is the best model.

8. Web Application

The interface uses a dark theme with a two-panel layout: the canvas on the left and the model output on the right. Below them are the model summary cards and the data charts.

8.1 Initial Screen

When the page loads, the canvas is empty and the output panel reads "Waiting for ink". The classifier drop-down starts with Random Forest, the brush slider is set to 30 px, and the Clear and Predict buttons are available. The "Model online" badge confirms that the models were loaded.

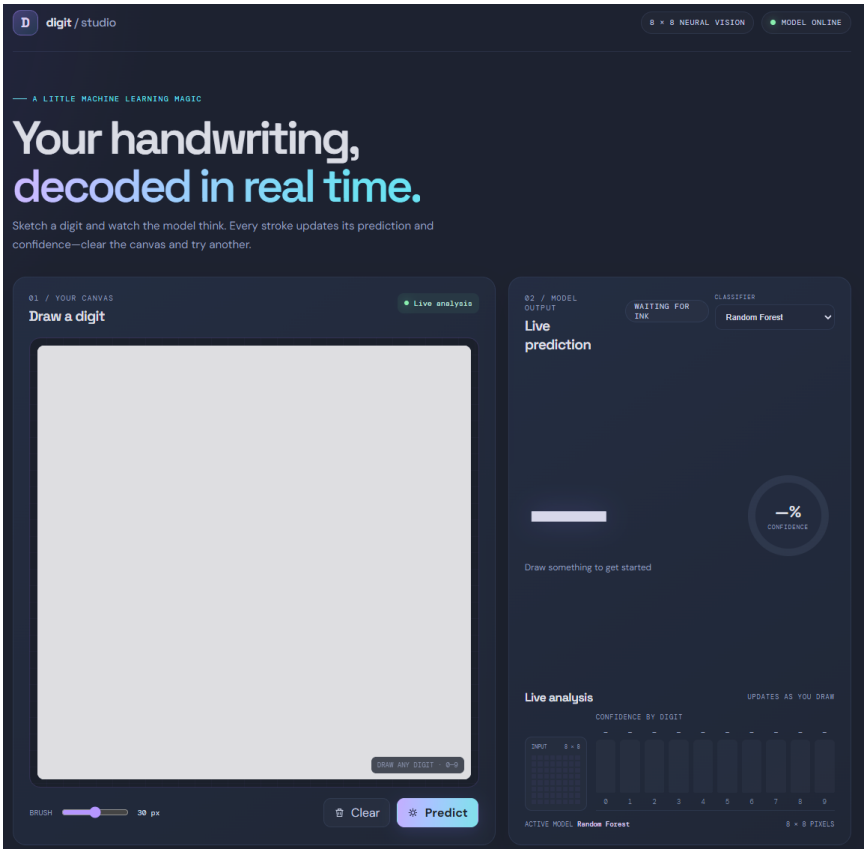


Figure 2: Home screen before drawing

8.2 Live Prediction

Live analysis updates the result as the user draws. In the example below, a “2” was drawn and the Random Forest predicted 2. Its confidence was only 39.0%, with 18% going to “8” and 10% to “3”. The 8 × 8 input preview in the bottom-left corner of the panel shows what the model received after preprocessing. This is useful to explain why a prediction may be uncertain: after a heavy brush stroke is reduced to 64 pixels, the shape loses detail.

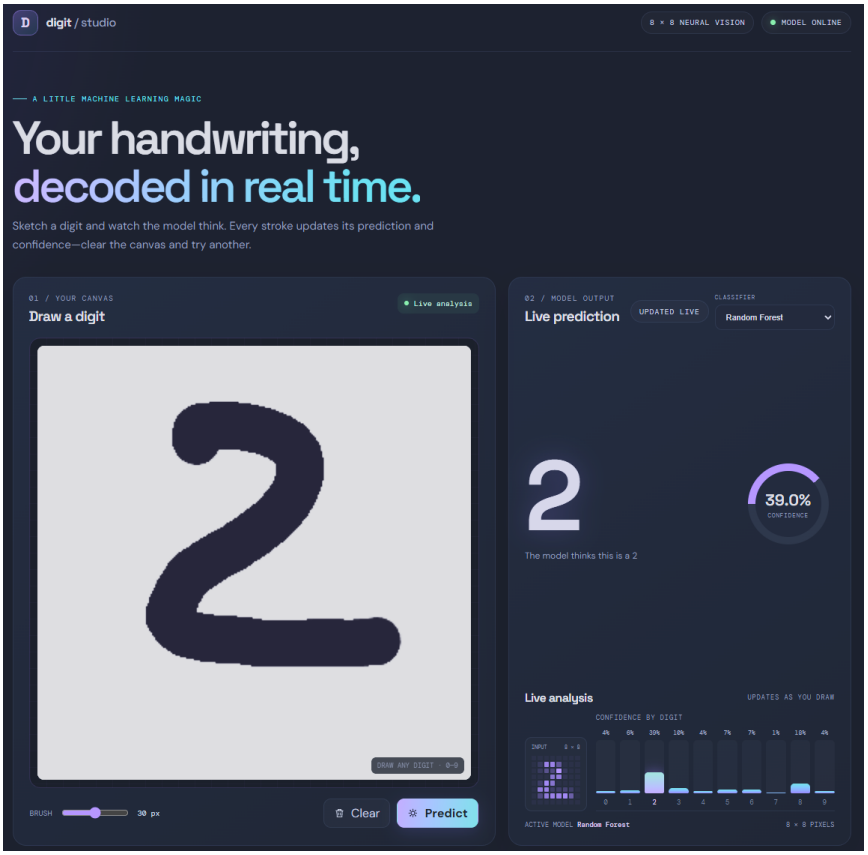


Figure 3: Prediction for a drawn “2” using the Random Forest

8.3 Majority Vote Mode

Besides the three models, the drop-down offers an experimental “Majority vote” option. Each model casts one vote and the percentage of votes becomes the score. In the example, a drawn “3” received two of three votes (66.7%) for 3, and the remaining vote went to 8. Because only three models vote, this score can only be 33%, 67% or 100%, and it measures agreement between models rather than a true probability.



Figure 4: Majority vote result for a drawn “3”

9. Input Validation and Security

Since the server accepts uploaded images, app.py applies several checks before decoding anything:

- Request body limited to 4 MB, and decoded image limited to 2 MB.
- Only base64 PNG data URLs are accepted, and base64 is strictly validated.
- Image format must really be PNG; width and height may not exceed 4096 pixels, and Pillow’s decompression-bomb protection is handled.
- The model name must be in the loaded model list or the majority-vote option, otherwise a 400 error is returned.
- Error messages are generic and do not reveal internal details.

10. Project Structure and Deployment

10.1 Folder Structure

The project folder (digit-app) is organized as shown in Figure 5, and its files have the following roles:

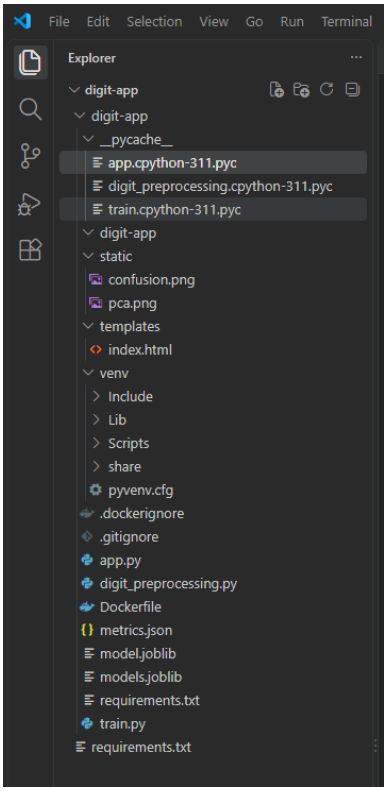


Figure 5: Project folder in VS Code

File	Purpose
app.py	Flask application: loads models, validates input, serves /, /predict and /health.
digit_preprocessing.py	Shared preprocessing and augmentation functions.
train.py	Trains and tunes models, saves joblib files, metrics.json and charts.
models.joblib / model.joblib	All three trained models / the best single model.
metrics.json	Accuracy results and best parameters shown on the page.
templates/index.html	Front-end page with canvas, controls and charts.
static/confusion.png, pca.png	Charts generated by train.py.
requirements.txt	Pinned Python dependencies.
Dockerfile, .dockerignore	Container build instructions and excluded files (venv, __pycache__, .git).
.gitignore	Files excluded from version control.

10.2 Docker Deployment

The Dockerfile starts from python:3.11-slim, installs the pinned requirements, copies the project and runs train.py during the image build, so the models and charts are always generated with the same library versions that serve them. The container listens on the port in the PORT variable (default 5000) and starts the app with Gunicorn.

```
docker build -t digit-studio .
```

```
docker run -p 5000:5000 digit-studio
```

For local development, the application can also be run with python app.py and opened at http://127.0.0.1:5000, as in the screenshots.

11. Limitations

- The dataset is small (1,797 images, 8 × 8 pixels), so fine details of handwriting are lost.
- Accuracy scores describe the dataset and computer-made variations; they are not measured on a large set of real users' drawings.
- Live predictions can show low confidence, for example 39% in Figure 3, especially for thick strokes or unusual digit shapes.
- The majority-vote score has only three possible non-zero values and is marked experimental.
- The scikit-learn digits set differs from larger datasets such as MNIST, so results should not be compared with MNIST benchmarks.

12. Future Scope

- Train on larger datasets such as MNIST (28 × 28) or EMNIST and compare with a convolutional neural network.
- Collect drawings from users (with consent) to measure real-world accuracy and retrain.
- Add probability calibration so confidence values are more meaningful.
- Support multi-digit recognition and touch/stylus pressure input.
- Add automated tests and a continuous-integration pipeline.

13. Conclusion

Digit Studio shows the full path from data to a usable product: preprocessing that makes canvas drawings compatible with the dataset, three tuned classifiers, augmentation for stroke variation, a Flask API with input validation, and an interactive front end that exposes the model's confidence and its 8 × 8 view of the input. The SVM reached 99.44% test accuracy and 97.22% on varied strokes, the best of the three models. The results also show the gap between clean-data accuracy and real drawings, a reminder that evaluation should match how a model will actually be used.